

Rubble_service feature Flag thoughts for design

Owners publish “**Materials Wanted**” with a per-unit offer (variable amount). Drivers pick “**What are you hauling?**”, and results show **only** sites actively seeking that material and passing rule checks (rebar/trash/etc.). Every received rubble/material **visit** charges the **owner** a **\$2 platform fee**.

UX: Simple, predictable, fast

Owner (Location editor → “Rubble Drop-Off”)

Section: “Materials Wanted”

- **Preset chips:** Dirt, Fill dirt, Asphalt, Brick.
- **Add custom:** free-text (“Crushed concrete 3–4in”).
- For each item (row):
 - **Pay rate:** amount + **unit** (per rock hauler trailer | per dump truck | per load | per ton | per cubic yard)
 - **Rules:** Rebar OK (Y/N), Trash/Wood (Y/N), Max piece size (in), Contaminants notes
 - **Capacity controls** (optional): Daily capacity, Queue max
 - **Active toggle**

Section: “General Rules”

- Hours, appointment only (Y/N)
- Site-wide constraints (applies to all materials unless overridden on a row)

Pricing note (always visible)

“\$2.00 platform fee is charged to your account for each material load received.”

Driver (Home / Dashboard)

Step 1: “What are you hauling?”

- Big selector cards with icons: Dirt, Fill dirt, Asphalt, Brick, **Custom...**
- If Custom... → quick picker or text (we’ll normalize; see below)

Step 2: Results list (flag-guarded)

- Only show locations with a **matching active material** *and* rules pass.
- List item shows:
 - Title + distance / ETA
 - **“Pays \$X / unit”** badge (or “Free” if rate = 0)
 - Rule badges: “Rebar OK/No”, “Trash OK/No”, “Max 8in”
 - Capacity/queue: “5 loads left today” | “Queue ~ 20 min”
 - CTA: “Navigate” / “Start drop-off”
- **Sort:** ETA asc → **Net payout** desc → Queue asc
(Net payout = owner pay for qty – driver surcharges if any; your current model is owner pays driver.)

Empty state:

“No active sites seeking ‘Asphalt’ within 25 mi. Try expanding radius or selecting another material.”

Data model (minimal, scalable)

1) Materials catalog (for presets + normalization)

```
materials
- id (PK)
- slug (unique, e.g., 'dirt', 'asphalt')
- display_name ('Dirt')
- synonyms JSON (['soil', 'topsoil'])
```

2) Owner wishes (materials wanted per location)

```
location_material_intents
```

- id (PK)
- location_id (FK -> locations)
- material_slug (FK -> materials.slug) // or null if custom
- custom_label TEXT // for custom entries
- unit ENUM('per_load','per_ton','per_cy')
- rate_cents INTEGER // amount owner pays to driver; 0 allowed
- rules JSON { rebar_ok: bool, trash_ok: bool, wood_ok: bool, max_piece_inches: int|null }
- capacity_daily INTEGER|null
- queue_max INTEGER|null
- active BOOL DEFAULT true
- updated_at TIMESTAMP

3) Unify visits (you likely already have this)

Add two fields and reuse your `service_visits`:

```
service_visits
- service_type ENUM('washout','rubble_dropoff')
- material_slug TEXT|null
- material_custom_label TEXT|null
- qty FLOAT, unit ENUM('load','ton','cy')
- amount_cents_owner_to_driver INTEGER
- fee_cents_platform INTEGER DEFAULT 0 // always 200 when
service_type='rubble_dropoff' and visit completes/arrives
- ... (photos_before/after, gps_check_ok, etc.)
```

Why this shape? You keep **one** “intent per row” per location, so search is a single indexed query, not a JSON filter. It also cleanly supports **multiple materials** per site.

Search & matching logic

Inputs: driver’s selected material (slug/custom), qty + unit (optional), current GPS, radius.

Filters:

1. Flag on (feature_rubble_dropoff_v1) and site active/open.

2. `location_material_intents.active = true`.
3. Material match:
 - If driver selected preset: match by `material_slug`.
 - If custom: normalize to a known slug via synonyms; if no match, use fuzzy LIKE on `custom_label`.
4. Rule checks:
 - If driver indicates **rebar present**, only show intents with `rebar_ok = true`.
 - Same for trash/wood; if driver doesn't specify, don't exclude.
5. Capacity/queue not exceeded.

Ranking:

- `ORDER BY drive_time ASC, rate_cents DESC, queue_estimate ASC`.

Edge behavior:

- If driver didn't specify rules (rebar/trash), show more results but badge conflicts ("Rebar not allowed") and require acknowledgment before "Start drop-off".

Pricing & \$2 platform fee

Owner offer = `rate_cents × qty`.

Platform fee = flat **200 cents** per **completed (or arrived) visit** charged to owner (debit).

Payout to driver = owner offer **minus** any additional platform % fee if you apply one later (right now, 0%).

Your revenue = \$2 per rubble/material visit (minimum).

Example:

- Owner pays \$15 per ton, driver drops 3.2 tons → `owner_to_driver = $48.00`, `platform_fee = $2.00` owner debit.

Implementation tip: book the \$2 on **arrive** (after geofence passed) to deter abuse; refund on cancel/time-out.

API (flag-aware)

Search

```
GET /rubble/search?
material=asphalt&lat=..&lng=..&radius=..&rebar=true&trash=f
alse
```

Returns array of `location_material_intents` with location data & computed ETA.

Create visit

```
POST /rubble/visits
```

```
{
  "intent_id": "lmi_123",
  "qty": 1.0,
  "unit": "per_load",          // 'per_load' | 'per_ton' | 'per_cy'
  "driver_declares": { "rebar": true, "trash": false }
}
```

Server:

- Validates rules & capacity.
- Computes `amount_cents_owner_to_driver`.
- Sets `fee_cents_platform` = 200 (to be booked on arrive).
- Creates `service_visit`.

Arrive & Complete

- `POST /rubble/visits/:id/arrive` → geofence check → **debit owner \$2** immediately; lock capacity.
- `POST /rubble/visits/:id/complete` → capture after photos; credit driver the owner offer.

UI components (what the user sees)

Owner editor (Rubble tab)

Materials Wanted (table)

- Row pattern:
[Material dropdown (chips + “Custom”)] [Amount] [Unit ▼] [Rebar OK ☐] [Trash OK ☐] [Wood OK ☐] [Max piece (in)] [Active ☐] (:)
- “+ Add material” button.
- Sticky note: “We charge a flat \$2.00 per received load.”

Driver dashboard

- **Step 1 card:** “What are you hauling?” (chips, scrollable)
- **Step 2:** Results list
 - Title (Location) – distance/time
 - **Pill:** “Pays \$X/unit”
 - Rule chips (Rebar OK/No, Trash OK/No, Max piece)
 - Capacity/queue info
 - Primary button: “Start drop-off”
- **Header filter bar:** Radius, Open Now, Appointment Only, Rebar toggle.

Normalization & custom materials (keeps lists clean)

- On custom entry, run a **lightweight normalizer**:
 - Lowercase, strip punctuation.
 - Map synonyms (e.g., “soil, topsoil → dirt”; “bitumen → asphalt”).
 - If a close match exists, store `material_slug` and keep `custom_label` for display.

- Else, store `material_slug = null, custom_label = 'Owner text'` and match by LIKE in search.

SQL (portable sketch)

```
CREATE TABLE materials (  
  id TEXT PRIMARY KEY,  
  slug TEXT UNIQUE NOT NULL,  
  display_name TEXT NOT NULL,  
  synonyms JSON  
);  
  
CREATE TABLE location_material_intents (  
  id TEXT PRIMARY KEY,  
  location_id TEXT NOT NULL REFERENCES locations(id),  
  material_slug TEXT REFERENCES materials(slug),  
  custom_label TEXT,  
  unit TEXT NOT NULL CHECK (unit IN  
( 'per_load', 'per_ton', 'per_cy' )),  
  rate_cents INTEGER NOT NULL DEFAULT 0,  
  rules JSON, --  
  {"rebar_ok":true,"trash_ok":false,"wood_ok":false,"max_piec  
e_inches":8}  
  capacity_daily INTEGER,  
  queue_max INTEGER,  
  active BOOLEAN NOT NULL DEFAULT true,  
  updated_at TIMESTAMP DEFAULT NOW()  
);  
  
-- Indexes for fast search  
CREATE INDEX idx_lmi_location_active ON  
location_material_intents(location_id, active);  
CREATE INDEX idx_lmi_material_slug ON  
location_material_intents(material_slug);  
CREATE INDEX idx_lmi_updated_at ON  
location_material_intents(updated_at);  
  
-- Visits already exist; add columns if needed
```

```
ALTER TABLE service_visits
  ADD COLUMN material_slug TEXT,
  ADD COLUMN material_custom_label TEXT,
  ADD COLUMN fee_cents_platform INTEGER DEFAULT 0;
```

Pricing calc (server)

```
function computeRubblePricing({ rate_cents, unit, qty }) {
  const base = Math.round(rate_cents * (qty ?? 1));
  const platformFlat = 200; // $2.00
  return {
    owner_to_driver_cents: base,
    platform_fee_owner_cents: platformFlat
  };
}
```

Apply platform fee on **arrive**. On cancel/no-show, void it and release capacity.

Feature flag wiring (what to gate)

- **Gated UI:** “What are you hauling?” selector, Rubble results, Rubble CTAs, Owner “Materials Wanted” section.
- **Gated APIs:** /rubble/*.
- **Kill switch:** ops_disable_rubble_dropoff blocks *new* visits but allows in-flight completes.

Safety & abuse controls (quick wins)

- Require **before/after photos**.
- Enforce **geofence** at arrive (200–300 ft) and a **time window** between arrive and complete.
- If driver declares rebar but intent rebar_ok=false → hard block.
- Per-driver strike count for rule violations → temporary block for that material type

